

Product Requirements Document (PRD)

AI-Powered Multi-Intent Cold Outreach Agent • Version 1.0.0 • Engineering & Product Specification

1. Executive Summary & Objective

The **AI Cold Outreach Agent** is a full-stack, enterprise-grade web application designed to automate personalized communication campaigns. Users upload tabular datasets (Excel/CSV) spanning multiple verticals—such as *Scholarship Applications*, *B2B Business Proposals*, and *App/Web Development Pitching*. The system utilizes Large Language Models (LLMs) to generate context-aware, hyper-personalized messaging and securely dispatches them via user-configured Gmail credentials with built-in rate limiting and safety protocols.

2. Target Use Cases & Data Mapping

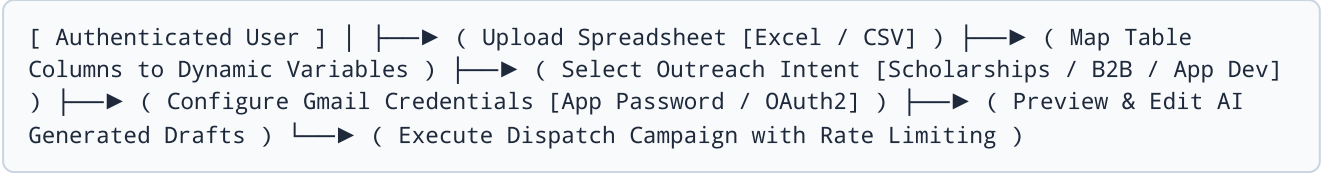
 Scholarship Outreach Columns: ApplicantName, University, Professor, FieldOfStudy, PastResearch. Objective: Draft persuasive academic inquiry letters to professors.	 App & Web Dev Cold Sales Columns: ClientName, Company, WebsiteAuditNote, TechStack. Objective: Pitch tailored digital transformation & code modernization services.
--	--

3. System Architecture & Tech Stack

Layer	Technology Choice	Core Responsibility
Frontend	Next.js (App Router), Tailwind CSS, shadcn/ui	Dashboard UI, dynamic column mapping, drag-and-drop file upload, live progress tracking.
Backend & AI	Next.js API Routes / Server Actions + Vercel AI SDK	Spreadsheet parsing (SheetJS), prompt engineering pipeline, LLM orchestration.
Database	PostgreSQL + Prisma ORM	Persistent session tracking, campaigns storage, audit logs for sent emails.
Transport	Nodemailer (SMTP over Google App Passwords)	Secure outbound email dispatch with randomized human-like delay intervals.

4. Use Case Diagram (Textual Representation)

The platform supports distinct actor roles interacting with core boundaries:



5. Class Diagram (Data Model Structure)

```
class User { String id String email String encryptedGmailAppPassword DateTime createdAt relation campaigns } class Campaign { String id String userId String title String intentType ("scholarship" | "b2b" | "web_dev") String customPromptTemplate relation recipients } class RecipientRow { String id String campaignId Json rawRowData // Dynamic columns from Excel String generatedSubject String generatedBody String status // PENDING, SENT, FAILED DateTime sentAt }
```

6. Non-Functional Requirements

- **Security & Privacy:** User Gmail credentials must be encrypted at rest using AES-256. Raw passwords are never logged.
- **Rate Limiting:** Enforce a randomized delay (15 to 45 seconds) between consecutive email dispatches to prevent SMTP blacklisting.
- **Fault Tolerance:** If an email dispatch fails due to network or SMTP limits, log the exact error message in `RecipientRow` and permit single-row retries.

7. Database Schema & Prisma ORM Definition

PostgreSQL relational schema required to track campaign states and execution queues:

```
datasource db { provider = "postgresql" url = env("DATABASE_URL") } model User { id String @id @default(cuid()) email String @unique gmailUser String gmailAppPass String // Encrypted string createdAt DateTime @default(now()) campaigns Campaign[] } model Campaign { id String @id @default(cuid()) userId String user User @relation(fields: [userId], references: [id]) title String intentType String promptTemplate String recipients RecipientRow[] createdAt DateTime @default(now()) } model RecipientRow { id String @id @default(cuid()) campaignId String campaign Campaign @relation(fields: [campaignId], references: [id], onDelete: Cascade) data Json // Dynamic Excel row key-values subject String? body String? status String @default("PENDING") // PENDING, SENT, FAILED errorLog String? sentAt DateTime? }
```

8. Core Database Queries & Operations

Key backend query routines executed during campaign generation and execution lifecycle:

```
-- Fetch pending campaign queue items for background dispatch worker SELECT r.id, r.data, c.promptTemplate, u."gmailUser", u."gmailAppPass" FROM "RecipientRow" r JOIN "Campaign" c ON r."campaignId" = c.id JOIN "User" u ON c."userId" = u.id WHERE r.status = 'PENDING' AND c.id = 'target_campaign_id' LIMIT 50; -- Update recipient row state post-dispatch execution UPDATE "RecipientRow" SET status = 'SENT', "sentAt" = NOW(), subject = $1, body = $2 WHERE id = $3;
```